Imperial College London
Department Of Electrical And Electronic Engineering

Autonomous Driving Robot

Mini-Group Project

ELEC70120

Fundamentals and Innovative Applications of Robotics

Authors:

Benjamin Gras

CID: 06058885

Ahmed Zekry

CID: 06047931

Andres Godoy

CID: 06053300

Submitted in partial fulfilment of the requirements for the MSc degree in Artificial Intelligence Applications and Innovation of Imperial College London

Spring 2025

Contents

1 Introduction **2**

2 Objectives **2**

3 Theoretical Background **2**

3.1 PID Control for Lane Following 2

3.2 HSV Colour Filtering 3

3.3 Object Detection with YOLOv4-tiny 3

3.4 Ackermann Steering Geometry 3

4 Hardware and Software Setup **4**

4.1 Robot Platform 4

4.2 Software Architecture 4

5 Methodology **5**

5.1 Navigation 5

5.1.1 Approach Selection 5

5.1.2 Lane Detection 6

5.1.3 Turn Detection and Counting 7

5.1.4 Open-Loop Navigation 8

5.1.5 Hybrid Architecture 8

5.2 Sign Detection 9

5.2.1 Model and Training 9

5.2.2 Inference and Integration 10

5.3 Parking 11

6 Results **12**

6.1 Experimental Setup 12

6.2 Challenges 13

7 Safety and Ethical Considerations **13**

8 Conclusion **14**

1 Introduction

The design, implementation, and evaluation of an autonomous driving system for a small-scale robot are described in this report. The robot must navigate a predefined circuit (S-1-2-3-9-4-5-6-8-3-9-6-E), follow yellow lane markings, react to traffic lights and road signs, and park at the end of the course—all without human intervention.
The project was decomposed into three subsystems—**navigation**, **sign detection**, and **parking**—developed in parallel before integration. For each subsystem, we conducted a comparative analysis of candidate approaches, testing multiple methods before selecting the one best suited to the task.
A central theme of our design is that **simplicity, when well-executed, produces the smoothest driving**. Rather than pursuing complex algorithms for their own sake, we systematically evaluated alternatives and selected the simplest approach that met each requirement. This philosophy was motivated by the assessment criteria, which heavily reward smooth and efficient motion, and penalise collisions, human interventions, and exits from the driving space.

2 Objectives

1. **Lane following**: autonomously follow yellow lane markings along the prescribed path.
2. **Turn handling**: execute both outside (wide) and inside (tight) turns reliably.
3. **Traffic sign detection**: detect and react to road signs in real time (go, right, crosswalk, parking).
4. **Traffic light reaction**: stop on red, proceed on green.
5. **Parking**: detect the parking sign and autonomously park at position E.

3 Theoretical Background

3.1 PID Control for Lane Following

A Proportional–Integral–Derivative (PID) controller minimises the lateral error between the robot and the lane centre. Given a deviation $e(t)$ from a target setpoint:

$$u(t) = K_p\, e(t) \;+\; K_i \int_0^t e(\tau)\, d\tau \;+\; K_d\, \frac{de(t)}{dt} \tag{1}$$

The proportional term corrects the current error, the integral term eliminates steady-state offset, and the derivative term dampens oscillations. In practice, the choice of which terms to include depends on the noise characteristics of the feedback signal and the dynamics of the system.

3.2 HSV Colour Filtering

Yellow lane markings are detected by converting frames from RGB to HSV (Hue–Saturation–Value) colour space. HSV decouples chrominance from luminance, making colour-based segmentation more robust to lighting variations than direct RGB thresholding. A binary mask is obtained with `cv2.inRange`, followed by morphological operations (erosion and dilation) to suppress noise.

3.3 Object Detection with YOLOv4-tiny

YOLOv4-tiny [1] is a lightweight single-stage detector from the Darknet framework. It processes the full image in one forward pass, predicting bounding boxes and class probabilities simultaneously. Its compact architecture (input 416×416, $\sim$23 MB weights, inference via OpenCV DNN) makes it suitable for real-time embedded deployment.

3.4 Ackermann Steering Geometry

The robot uses Ackermann steering. The relationship between angular velocity ω and steering angle α is:

$$\omega = \frac{v \cdot \tan(\alpha)}{L} \tag{2}$$

where v is the linear velocity and $L = 0.213\,\mathrm{m}$ is the wheelbase. This kinematic model converts controller outputs into velocity commands throughout the system.

4 Hardware and Software Setup

4.1 Robot Platform

Figure 1: The Hiwonder JetAcker robot used in this project.

The robot is a Hiwonder JetAcker, an Ackermann-steering platform equipped with:

- **RGB-D camera**: colour images (640×480) and depth data.
- **2D LiDAR**: 360° laser scanner.
- **Wheel encoders**: provide velocity feedback for distance estimation.
- **Ackermann steering**: front servo, wheelbase $L = 0.213\,\mathrm{m}$.

4.2 Software Architecture

The system runs on ROS 1 [2]. Figure 2 shows the node architecture.

ROS Node Architecture

`Camera` $\xrightarrow{\text{image}}$ `Navigation node` $\xrightarrow{\text{cmd_vel}}$ **Motors**

`Camera` $\xrightarrow{\text{image}}$ `YOLOv4-tiny` $\xrightarrow{\text{detections}}$ `Navigation node`

`LiDAR` $\xrightarrow{\text{scan}}$ `Safety controller` $\xrightarrow{\text{override}}$ **Motors**

Figure 2: Simplified ROS node and topic architecture.

The main custom components are:

- **Navigation node**: lane following, turn counting, open-loop navigation, and parking orchestration.
- **Lane detection module**: HSV-based lane detection with configurable ROI bands.
- **Sign detection node**: YOLOv4-tiny inference; publishes detected sign labels.
- **Safety controller**: independent LiDAR-based emergency stop (see Section 7).

5 Methodology

The methodology is organised around the three subsystems: navigation (Section 5.1), sign detection (Section 5.2), and parking (Section 5.3).

5.1 Navigation

5.1.1 Approach Selection

Before committing to an implementation, we invested significant effort in evaluating candidate approaches. The goal was not to implement the most complex solution, but to find the approach that would maximise **driving smoothness and reliability**—the two factors most heavily weighted in the assessment.

Candidate approaches. We considered four methods:

Approach	Advantages	Disadvantages
SLAM / Mapping	Accurate, reusable map; robust localisation	Heavy compute; overkill for a fixed track; less fluid motion
Reinforcement Learning	Learns optimal policy end-to-end	Long training; sim-to-real gap; no simulator for this track
Pure Odometry	Simple; no vision needed	Drift accumulates on straights; no feedback
Visual Lane Following	Reactive, lightweight, closed-loop	Fails when line exits field of view (inside turns)

Table 1: Comparative evaluation of navigation approaches.

Experimental evaluation. We did not select our approach based on theoretical analysis alone—we prototyped and tested multiple methods on the physical test mat:

- **SLAM**: we built a full occupancy-grid map of the test site using the robot's LiDAR and the on-board SLAM package (see Figure 3). While this produced an

accurate map, the resulting navigation was less fluid than direct lane following: the path planner introduced unnecessary corrections and the motion appeared jerky compared to a simple PID following a visible line. Additionally, the computational overhead was significant for an embedded platform.

- **Pure visual lane following**: we implemented a full PID controller (K_p=0.005, K_i=0.0001, K_d=0.001) tracking the yellow lane centroid. This produced *very smooth* straight-line driving. However, it failed on inside turns: the yellow line exits the camera's field of view, the robot loses its reference, and the PID oscillates or drives off-track.

- **Pure odometry**: we tested timed manoeuvres for the full circuit. Turns were handled well, but on straights the robot drifted due to the absence of visual feedback, requiring frequent recalibration.

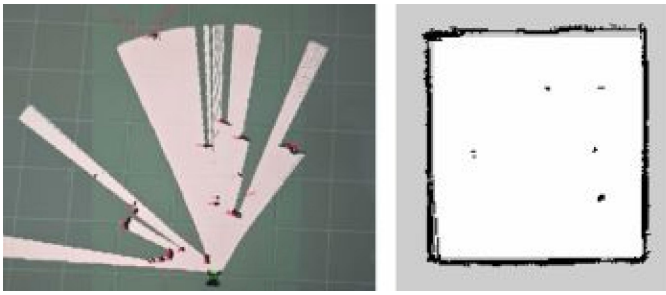


Figure 3: Occupancy-grid map (right) built via SLAM during the exploration phase (left), showing the track layout.

Conclusion. Each method had a clear strength and a clear weakness. Visual lane following gave the smoothest driving on straights; odometry gave reliable turns. This led us to a **hybrid architecture**: use vision where it works best (outer circuit, where the line is always visible) and switch to open-loop navigation where vision fails (inner circuit, with tight turns). The key insight from our testing was that *the simplest approach that works is also the smoothest*, because additional complexity (SLAM path planning, PID fighting a noisy signal) introduces micro-corrections and hesitations that degrade fluidity.

5.1.2 Lane Detection

The lane detection module processes each camera frame through the following pipeline:

1. **RGB** $\rightarrow$ **HSV** conversion.

2. **Gaussian blur** (3×3) to reduce high-frequency noise.

3. **Colour mask**: `cv2.inRange` for yellow ($H \in [20, 40]$, $S \in [80, 255]$, $V \in [80, 255]$).
4. **Morphological filtering**: erosion then dilation (3×3 rectangular kernel).
5. **ROI extraction**: three horizontal bands at different image heights, weighted by proximity to the robot (70% / 20% / 10%).
6. **Centroid extraction**: for each band, the largest yellow contour is found via `cv2.findContours`; its centre x-coordinate is extracted. The weighted sum yields the `lane_x` value.

```
# (row_start, row_end, col_start, col_end, weight)
LEFT_ROIS  = ((450, 480, 0, 320, 0.7),    # bottom-left (closest)
              (390, 420, 0, 320, 0.2),     # mid-left
              (330, 360, 0, 320, 0.1))     # upper-left (farthest)
RIGHT_ROIS = ((450, 480, 320, 640, 0.7),
              (390, 420, 320, 640, 0.2),
              (330, 360, 320, 640, 0.1))
```

Code 1: ROI definitions for left-side and right-side lane following.

The 70/20/10 weighting prioritises the closest band for immediate steering feedback, while the upper bands provide early warning of upcoming curves. These weights and the HSV thresholds were the parameters that required the most empirical testing—calibrating them for reliable detection across lighting conditions was one of the most time-consuming aspects of the project.

Proportional-only control. The `lane_x` value is fed to a proportional controller:

$$\omega = \frac{v \cdot \tan\big(\text{clamp}(K_p \cdot (\texttt{lane_x} - \texttt{setpoint}),\ -0.1,\ 0.1)\big)}{L} \tag{3}$$

with $K_p = 0.01$, setpoint $= 100\,\text{px}$ (left-following) or $620\,\text{px}$ (right-following), and constant $v = 0.15\,\text{m/s}$. During prototyping, we observed that reducing the controller from full PID to proportional-only *improved* smoothness: the integral term accumulated error during turns and caused overshoot on re-entry to straights, while the derivative term amplified the discrete, noisy `lane_x` signal. The proportional term alone, with a well-chosen gain, provided sufficient correction for the gentle deviations encountered on straight segments. The clamp at ± 0.1 further limits the steering range, preventing aggressive corrections.

5.1.3 Turn Detection and Counting

The robot tracks its position on the circuit by counting turns. Turn detection is based on the displacement of the `lane_x` centroid:

- **Outside turns** (e.g., following the left line and turning right): the yellow line shifts towards the edge of the frame. A turn is detected when `lane_x` exceeds a calibrated threshold.
- **Inside turns** (line disappears towards the turn centre): detected when `lane_x` drops below the threshold or the line is lost (`lane_x` $= -1$).

The turn counter serves as a simple localisation mechanism: after the third outside turn, the robot knows it has reached the inner circuit and transitions to open-loop navigation.

5.1.4 Open-Loop Navigation

For the inner circuit, where inside turns cause the yellow line to leave the camera's field of view, the robot switches to open-loop navigation using pre-calibrated manoeuvres.

Distance and time-based control. Each manoeuvre is defined by a commanded velocity v and a duration t. The estimated distance is $d = v \times t$. For straight segments, the robot drives a calibrated distance at constant speed. For turns, it applies a fixed steering angle for a calibrated duration:

1. **Backup**: reverse at $-0.15\,\mathrm{m/s}$ for $\sim$4 s to reposition behind the turn apex.
2. **Forward entry**: drive straight for $\sim$2 s to enter the turn zone.
3. **Forced steer**: apply a fixed steering angle ($\alpha = \pm 0.6\,\mathrm{rad}$) for $\sim$4.5 s to complete the turn.

Reproducibility as implicit localisation. A key observation from our testing was that after each turn, the robot consistently ended up at the same position (within a small tolerance). This *reproducibility* of post-turn positions effectively creates a virtual map: even without explicit localisation, we can predict where the robot is after each manoeuvre and plan the next one accordingly. This is what allows the open-loop approach to work reliably across the full inner circuit—each manoeuvre starts from a known position, so the cumulative drift remains bounded.
The durations and speeds for each manoeuvre were calibrated individually on the test mat. While this makes the system specific to the track, it produces **deterministic, smooth motion**—the robot commits to each manoeuvre without hesitation or oscillation.

5.1.5 Hybrid Architecture

The full circuit combines both navigation modes, transitioning between them based on the turn counter:

Navigation Flow

Phase 1 — Outer circuit (S → checkpoint 5): Visual lane following.
The robot follows the yellow line using the proportional controller. Outside turns are handled by switching to a fixed steering angle until `lane_x` returns to its normal range. The turn counter increments at each turn.

Phase 2 — Inner circuit (5 → 8): Open-loop navigation.
After the third outside turn, the robot switches to pre-calibrated manoeuvres (backup, forward, forced steer). Each manoeuvre starts from the reproducible end-position of the previous one.

Phase 3 — Return to outer (8 → 3): Visual lane following for one outside turn.

Phase 4 — Approach parking (3 → E): Open-loop navigation to reach the parking zone.

Phase 5 — Parking: Closed-loop visual parking (see Section 5.3).

Figure 4: Navigation phases across the circuit S-1-2-3-9-4-5-6-8-3-9-6-E.

The transitions between vision and open-loop modes are seamless: the robot finishes a visually-guided straight, detects a turn via the `lane_x` threshold, and immediately begins the calibrated manoeuvre. There is no pause or mode-switch delay.

5.2 Sign Detection

5.2.1 Model and Training

Traffic signs are detected using YOLOv4-tiny (Darknet) trained on 12 classes. The classes actively used during the demonstration were: *go, right, park, red, green, crosswalk*.

Parameter	Value
Input resolution	416×416
Batch size	64 (subdivisions: 16)
Max batches	26,000
Learning rate	0.00261 (step decay at 20,800 and 23,400)
Augmentation	Saturation 1.5, exposure 1.5, hue 0.1, jitter 0.3
Inference backend	OpenCV DNN
Weight file	$\sim$23 MB

Table 2: YOLOv4-tiny training configuration.

Dataset. The training dataset consisted of **965** annotated images split into **643** training and **322** validation samples, covering 10 classes of traffic signs and lights. Data was collected on the test site using a custom ROS node that captures frames when the visual similarity between consecutive images drops below a threshold, avoiding

redundant captures. To improve generalisation, the model was first **pre-trained on publicly available traffic sign datasets**, then fine-tuned on our domain-specific images. Standard data augmentation was applied during training (saturation and exposure jitter $\times 1.5$, hue jitter ± 0.1), as specified in the Darknet configuration. The model was trained for 26 000 iterations with a batch size of 64 using the CIoU loss, a learning rate of 2.61×10^{-3} with step decay at iterations 20 800 and 23 400. The final model achieved a mean average precision (mAP@0.5:0.95) of **83.6%** on the validation set.

5.2.2 Inference and Integration

At inference time, the camera image is cropped to the lower 75% (removing the ceiling region) before being passed to the model. Only detections with confidence ≥ 0.80 are retained; when multiple signs are detected simultaneously, the one with the **largest bounding-box area** (i.e., closest to the robot) is selected. Detected labels are published on a ROS topic and consumed by the navigation node to trigger the following actions:

- `red` (traffic light): the robot stops and enters a detection loop—it waits briefly, re-runs inference, and only proceeds once the red light is no longer detected.
- `green` / `go`: proceed at normal speed.
- `stop`: the robot stops for a fixed duration, then resumes.
- `speed limit`: when detected (typically before a turn), a boolean flag is set. The speed reduction is applied at the exit of the next turn, avoiding abrupt deceleration mid-manoeuvre.
- `park`: initiate the parking sequence (Section 5.3).
- `right`: confirm right turn direction.

Camera orientation for traffic lights. A practical challenge was that the camera's limited field of view prevented it from seeing traffic lights when the robot was driving straight towards them. Since the traffic lights are encountered during the odometry-controlled (open-loop) phases—where the camera is not needed for lane detection—we solved this by **rotating the camera servo** to point towards the traffic light before reaching it. This simple adjustment significantly improved detection reliability without affecting navigation.

Figure 5: YOLOv4-tiny detection output showing bounding boxes and confidence scores.

5.3 Parking

For parking, the robot uses a visual servo loop driven entirely by the YOLO parking sign detection. No odometry or pre-programmed sequence is used. Every decision is reactive, based solely on where the sign appears in the image.

Feedback signals: The robot reads two things from the bounding box:

- The horizontal centre c_x of the sign, which tells us lateral alignment.
- The area A of the box, which tells us how far away we are.

Targets: We have two targets. Centre-X should be around 78 pixels, meaning the sign is in the left portion of the frame, and area should be around 10 000 pixels squared, corresponding to the correct parking distance.

$$c_x^{\text{target}} = 78 \pm 8 \text{ px}$$
$$A^{\text{target}} = 10\,000 \pm 1\,500 \text{ px}^2$$

If both are within tolerance, the robot declares itself parked and stops.

Correction priority: If not, it corrects in priority order, lateral first, then distance. Approaching the sign while still misaligned would risk colliding with the spot boundary, so the robot only begins distance corrections once $|c_x - c_x^{\text{target}}| \leq 8$ px.

Lateral shift manoeuvre: For lateral correction it executes a 4-step parallel shift manoeuvre. This gives a pure lateral translation without changing heading. Left and right shifts are symmetric, so only the right shift is described:

1. **Steer right and advance:** Steering angle +0.45 rad, linear velocity 0.06 m/s, duration 0.6 s. The front wheels swing the nose to the right.

2. **Go straight to slide the rear axle across:** Steering angle 0, linear velocity 0.06 m/s, duration 0.3 s.

3. **Counter-steer left and advance to straighten up:** Steering angle −0.45 rad, linear velocity 0.06 m/s, duration 0.6 s.

4. **Reverse straight to cancel the net forward movement:** Steering angle 0, linear velocity −0.06 m/s, duration 0.9 s.

Each cycle gives a net lateral displacement of around 4 to 6 cm with negligible heading change.

Distance adjustment: For distance, it simply drives a short burst forward or backward:

- If $A < A^{\text{target}} - 1500$ (sign too small, too far away): drive forward at 0.05 m/s for 0.4 s.
- If $A > A^{\text{target}} + 1500$ (sign too large, too close): drive backward at 0.05 m/s for 0.4 s.

Convergence: This loop runs for up to 20 iterations, each time pausing briefly (around 0.2 s) to get a fresh camera reading before deciding the next correction. In typical conditions, convergence is achieved in 6 to 10 iterations: roughly 4 to 7 lateral cycles followed by 2 to 3 distance bursts. The 20-iteration cap is just a safety bound to prevent infinite loops if the sign detection becomes unstable. If the cap is reached, the robot stops in place and logs a warning.

6 Results

6.1 Experimental Setup

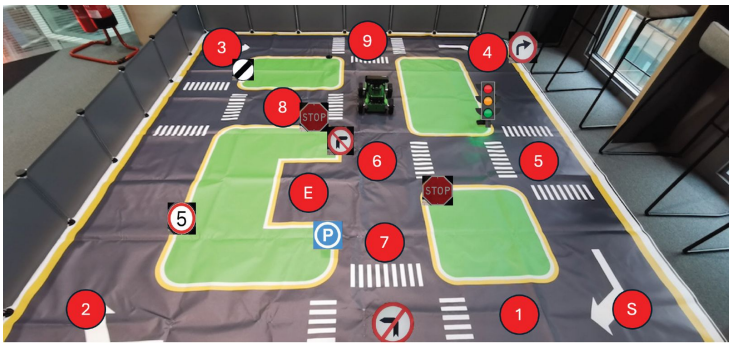
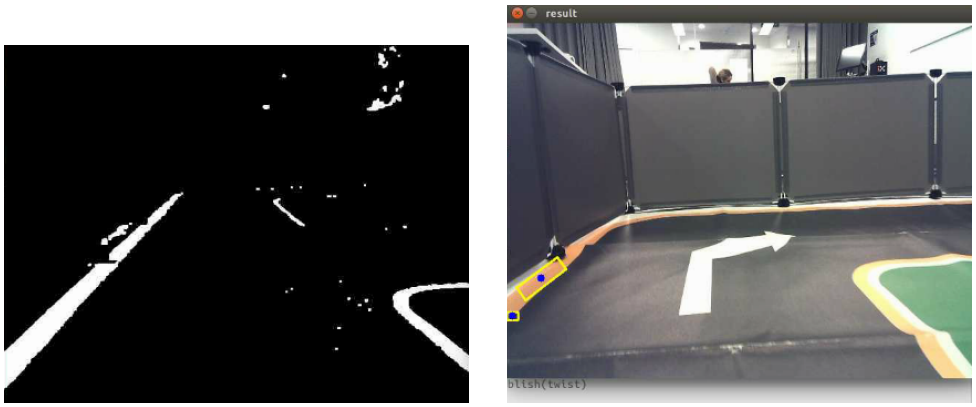


Figure 6: Test site with the prescribed navigation path S-1-2-3-9-4-5-6-8-3-9-6-E.

The system was tested on the official test site: a floor mat with yellow lane markings, traffic signs at designated positions, and a parking zone at position E. The robot operated at a constant forward speed of 0.15 m/s during normal driving.

6.2 Challenges



(a) HSV mask (b) Detected lane overlay

Figure 7: Yellow lane detection in real time: HSV mask (left) and detected lane overlay (right).

The most time-consuming aspect of development was **HSV threshold calibration**. Small changes in ambient lighting (time of day, nearby screens, overhead lamps) shifted the optimal bounds for yellow detection. As seen in figure 7 a, the picture on the left representing the yellow mask for the lane can sometimes become noisy resulting in poor lane detection which is observed in figure 7 b. This usually happens when the robot takes a turn, as it takes a second for it to recalibrate and focus due to lighting changes, leading to losing track of the lane at that time. We addressed this by choosing conservative thresholds that worked reliably across observed conditions, at the cost of occasionally missing faint line segments at the edge of the detection range. An adaptive approach (e.g., histogram-based calibration) would reduce this sensitivity. In addition, lane forecasting using training visual data could make the robot more robust to lighting changes, allowing the robot to stay on track even when the lane is noisy at an instance.

7 Safety and Ethical Considerations

Deploying autonomous robots in shared physical spaces raises important safety and ethical concerns, even in a controlled demonstration setting.

Layered safety architecture. Our system includes an independent safety controller that monitors the LiDAR over a 60° forward arc. If any valid range measurement falls

below 0.4 m, the controller overrides the velocity command to zero—regardless of what the navigation node commands. This *defence-in-depth* approach ensures that a software bug in the navigation logic cannot cause a collision.

```
for i in range(0, safety_angle):  # safety_angle = 60 degrees
    index = ((360 - safety_angle // 2) + i) % 360
    if range_max >= ranges[index] >= range_min:
        valid_ranges.append(ranges[index])

if min(valid_ranges) < 0.4:  # safety_distance = 0.4m
    override forward velocity to zero  # backward still allowed
```

Code 2: Safety controller: obstacle detection over the forward arc.

Predictability. The deterministic nature of our architecture—proportional control on straights, fixed manoeuvres on turns—makes the robot's behaviour interpretable and predictable. An observer can anticipate what the robot will do at each point on the circuit. This transparency is an important property for systems that operate near humans.

Failure modes. When the line is lost unexpectedly, the robot stops rather than guessing. The hardware failures encountered during development highlight the importance of robust system monitoring: in a production system, watchdog timers and graceful degradation mechanisms would be essential.

Scale gap. While our approach works well for a controlled demonstration, real autonomous driving requires handling dynamic environments, unpredictable actors, adverse weather, and regulatory compliance. The gap between demonstrations and deployment should not be underestimated. Our project illustrates both the promise of simple, well-tuned systems and the limitations of approaches that depend on a known, static environment.

8 Conclusion

We presented a hybrid autonomous driving system that combines visual lane following for the outer circuit with open-loop navigation for the inner circuit. Traffic signs are detected in real time by YOLOv4-tiny, and a closed-loop visual feedback sequence handles parking. Our key design principle was to **select the simplest approach that maximises driving smoothness**. This was not a shortcut but a deliberate engineering decision, informed by systematic testing of alternatives: we built a SLAM map, implemented full PID control, and tested pure odometry before converging on the hybrid architecture. Each candidate was evaluated not only on correctness but on *fluidity of motion*. The resulting system demonstrates that simplicity, when backed by rigorous experimentation, produces better results than unjustified complexity.

Scope and future directions. Our approach is well-suited to a fixed, known circuit: the open-loop navigation is reliable because the environment is static and the robot's post-turn positions are reproducible, while visual lane following provides smooth closed-loop control on straights. For more complex scenarios—variable track layouts, dynamic obstacles, changing surface conditions—the system would need to evolve:

- Adaptive HSV thresholds or ML-based lane detection for robustness to lighting and surface changes.
- Closed-loop turns using IMU heading feedback, replacing timed manoeuvres.
- Vision-based intersection detection for self-localisation, removing the dependency on a hardcoded route.
- SLAM-based navigation or dynamic replanning for unknown or changing environments, accepting the trade-off in motion fluidity.

References

[1] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[2] M. Quigley *et al.*, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, vol. 3, 2009.

[3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.